AI Meeting Notes Analyzer using LangGraph

Project Documentation

Author: Simran **AI Assistance:** This project was built with the help of Claude (Anthropic), which was used as a learning and pair-programming assistant throughout development — explaining LangGraph concepts, reviewing code, helping debug environment issues, and guiding design decisions. All code was written, tested, and run by me, in my own development environment.

Live App: https://ai-meeting-notes-analyzer-sihag.streamlit.app/ **GitHub Repository:** https://github.com/Simran280199/AI-Meeting-Notes-Analyzer

---

Table of Contents

---

1. Introduction

This document explains how I built the **AI Meeting Notes Analyzer**, a capstone project that takes a raw meeting transcript and turns it into a structured, readable report — with the key topics discussed, a short summary, the action items assigned to each person, and an overall priority level for the meeting. The whole thing is powered by a multi-agent pipeline built using **LangGraph**, with OpenAI's GPT models doing the actual language understanding, and a **Streamlit** web app on top so anyone can use it without touching the code.

I'm writing this document the way I actually built the project — step by step, in the order I actually did things, including the mistakes and dead ends along the way, because I think that's a more honest and more useful record than a polished after-the-fact summary. Meetings generate a lot of unstructured conversation, and by the end of a long call it's genuinely hard to remember exactly who agreed to do what. That's the real-world problem this tool solves, and it's also a good excuse to learn how modern LLM-based agent systems are actually built in practice, not just how to call a chat API once.

2. Problem Statement

Meetings are recorded or transcribed constantly — for status updates, sprint planning, customer support escalations, and so on — but almost nobody goes back and reads a full transcript afterward. What people actually want is much smaller: what did we talk about, what did we decide, who owes what, and how urgent is it. Manually extracting that from a transcript is tedious and error-prone, especially in longer meetings with several speakers talking over shared context, interruptions, and casual phrasing rather than clean, structured notes.

The specific transcript I used to stress-test this project — a team debugging an Android app crashing on login — is a good real-world example. Four different roles (Product Manager, Customer Support Lead, Mobile Developer, QA Tester, Backend Developer) are talking, nearly every person commits to at least one task, and the urgency is implied through phrasing ("please prioritize this issue today") rather than stated outright. A useful tool needs to correctly parse all of that.

3. Objectives

Before writing any code, I laid out what the finished project needed to actually do:

- Accept a raw meeting transcript as input (pasted text or an uploaded `.txt` file)
- Identify the main topics discussed
- Produce a short, readable summary of the meeting
- Extract every action item along with who is responsible for it
- Classify the overall urgency of the meeting's action items as High, Medium, or Low
- Correctly handle meetings that have **no** action items at all, without breaking or showing meaningless output
- Present all of this in a clean, usable interface — not just a printed block of text in a terminal
- Be properly documented and deployed, not just working on my own machine

That last point mattered to me — a project that only runs locally on one laptop isn't really finished.

4. Why LangGraph? Why Not a Single Prompt?

The simplest possible version of this project would be one big prompt: "Here's a transcript, give me the topics, summary, action items, and priority, all at once." I actually considered this first, and it's worth explaining why I didn't build it that way.

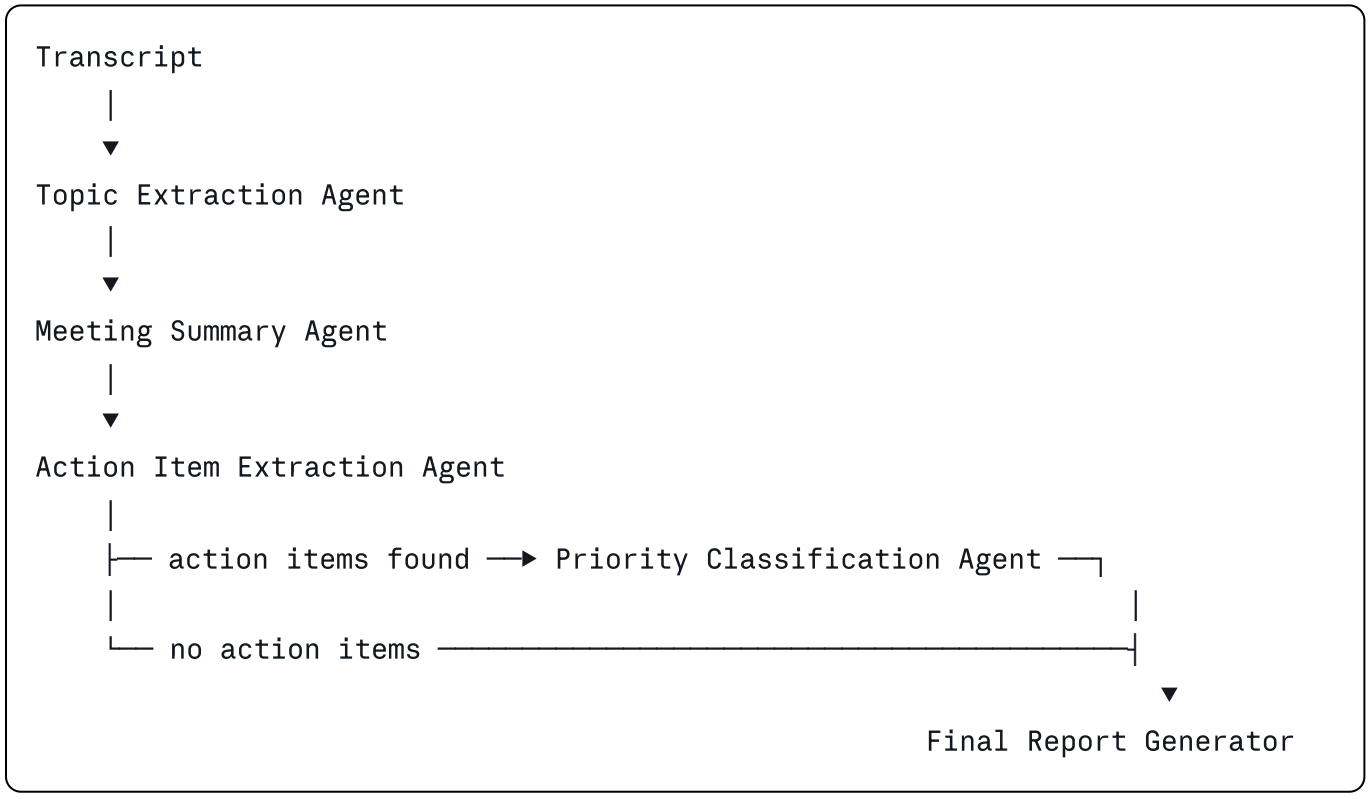
The problem with one giant prompt is that it collapses four genuinely different tasks — extraction, summarization, structured parsing, and classification — into a single request, which makes the system hard to control and hard to debug. If the action items come out wrong, I can't isolate and fix just that part without risking a ripple effect on the summary or topics, since they're all being generated together in one model call. It also makes conditional behavior awkward — I specifically needed the pipeline to skip priority classification entirely when there are no action items, and that kind of branching logic doesn't fit cleanly inside a single prompt.

LangGraph solves this by letting me treat the whole problem as a graph: small nodes (each one a focused function with one job), connected by edges that define the order they run in, including conditional edges that can route execution down different paths depending on what earlier nodes produced. Every node reads from and writes to one shared **state** object that flows through the whole pipeline — I ended up thinking of this state as a relay-race baton, since that mental model made the whole architecture click for me early on.

This decomposition is also just a better reflection of how production LLM systems are actually built — as coordinated small agents with clear responsibilities, not one prompt trying to do everything.

5. System Architecture

At a high level, the pipeline looks like this:



Every box except the last one calls the LLM once, with a narrow, specific prompt. The conditional check happens right after action item extraction: if the extracted list is empty, the graph skips straight to the final report, and the report substitutes a clear fallback message instead of an empty or nonsensical priority classification.

6. Tech Stack

- **Python** — the implementation language for the whole project
- **LangGraph** — orchestrates the agent pipeline: nodes, edges, and the one conditional branch
- **LangChain / langchain-openai** — the connector layer that lets LangGraph talk to OpenAI's models
- **OpenAI GPT (gpt-4o-mini)** — the underlying language model doing the actual reading and reasoning over the transcript
- **python-dotenv** — loads the API key from a local `.env` file, so it's never hardcoded into the source code
- **Streamlit** — the web interface, including a custom dark theme
- **Git and GitHub** — version control and hosting
- **Streamlit Community Cloud** — deployment, so the app is reachable at a public URL rather than only running on my own laptop

I chose `gpt-4o-mini` specifically because the tasks involved (extraction, summarization, classification) don't need the most powerful available model — they need consistency and speed, and a smaller, cheaper model does that well. I also set `temperature=0` on the model everywhere, since I wanted deterministic, repeatable output rather than creative variation — a meeting summary shouldn't read differently every time you regenerate it from the same transcript.

7. Environment Setup — What I Did and What Went Wrong

I want to document this honestly because it's a real lesson, not just a footnote. I initially tried to reuse a virtual environment (`venv`) copied over from a previous, unrelated project, rather than creating a fresh one. It looked like it worked — the terminal correctly showed `(venv)` active — but when I ran `pip install -r requirements.txt`, the installation logs revealed every package was actually being installed into the **old** project's folder path, not the new one.

The reason is that a Python virtual environment isn't just a folder of packages — the activation scripts and `pyvenv.cfg` inside it have the environment's original folder path baked in at creation time. Copying the folder to a new location doesn't update those internal paths, so the "copied" venv was secretly still pointing back at its original home. This is a well-documented Python gotcha (venvs are not relocatable), and the fix was straightforward once I understood the cause: delete the copied venv entirely and build a fresh one with `python -m venv venv` inside the actual project folder, then reinstall dependencies from `requirements.txt`.

I'm including this because I think it's a genuinely common trap for anyone reusing setup across projects, and understanding *why* it broke, rather than just applying a fix blindly, was one of the more useful debugging lessons in the whole project.

8. State Design — The Shared "Baton"

Before writing a single agent, I defined the data structure that would flow through the entire graph — `MeetingState`, implemented as a Python `TypedDict`:

```python
class MeetingState(TypedDict):
    transcript: str
    topics: List[str]
    summary: str
    action_items: List[Dict[str, str]]
    priority: str
    final_report: str
```

I made a couple of deliberate design decisions here that shaped everything downstream:

- `action_items` is a list of dictionaries (`{"task": ..., "owner": ...}`), not a list of plain strings. This keeps the task and the responsible person structurally separate from the very beginning, so the final report doesn't have to re-parse a sentence to figure out who owns what.
- `priority` is a single string representing the overall urgency of the meeting's tasks, not a per-item priority. This matched how the project brief described the requirement, and it also kept the final report simpler and more readable — one priority line, not a separate rating next to every task.

Designing this first, before any agent code, meant every node I wrote afterward had a clear, fixed contract for what it reads and what it's responsible for writing back.

9. Building the Agents, One by One

I built and tested each agent in isolation before connecting anything, which turned out to be the right call — when something didn't work, I always knew exactly which piece to look at.

Topic Extraction Agent. Reads the transcript, asks the model for a numbered list of short topic phrases, and parses the response into a real Python list using a list comprehension that strips the numbering off each line. The prompt explicitly says "return ONLY a numbered list, nothing else" — without that constraint, the model tends to add a friendly preamble like "Sure, here are the topics:" which would otherwise break the parsing.

Meeting Summary Agent. Writes a short, plain-prose summary. I explicitly told the model not to include action items or headers in this step, because without that instruction it tends to blend the summary and the task list together, since it can see the whole transcript and doesn't inherently know these are meant to be separate outputs handled by separate agents.

Action Item Extraction Agent. This one needed a more careful prompt, since the output has to become structured data (task + owner), not just a list of sentences. I asked the model

to return each item as `Task | Owner`, using a pipe character as a delimiter specifically because it's unlikely to appear naturally inside a task description, unlike a comma or period. I also asked for the literal word `NONE` if there were truly no action items, so the code has an unambiguous signal to check against, rather than guessing based on an empty or malformed response.

Priority Classification Agent. Takes the transcript and the already-extracted action items, and asks the model to return a single word — High, Medium, or Low — representing the overall urgency. This agent only ever needs to run when action items actually exist, which is exactly what the conditional logic (next section) enforces.

Final Report Generator. Unlike the other four, this node makes no LLM call at all. By the time execution reaches it, every other agent has already done its job — this node's only responsibility is formatting everything into one clean report string, including handling the no-action-items case with a clear fallback message rather than an empty or confusing section.

10. The Conditional Logic

The one branching decision in the whole pipeline happens right after action item extraction, using a small router function:

```python
def check_action_items(state: MeetingState) -> str:
    if state["action_items"]:
        return "has_items"
    else:
        return "no_items"
```

This function is different from the other five — it doesn't do any real work and it doesn't return state. Its only job is to look at what the Action Item Extraction Agent produced and hand back a short string label. LangGraph uses that returned label to decide which node runs next: `"has_items"` routes to priority classification, `"no_items"` skips straight to the final report. I originally found this a little confusing — I kept expecting it to behave like the other nodes — until I understood it as a fork in the road rather than a stop on it: it sits *between* two nodes, not on the graph as its own processing step.

11. Wiring the Graph

With all six functions written and individually tested, `graph.py` connects them into an actual `StateGraph`:

```python
```

```
builder = StateGraph(MeetingState)

builder.add_node("extract_topics", extract_topics)
builder.add_node("summarize_meeting", summarize_meeting)
builder.add_node("extract_action_items", extract_action_items)
builder.add_node("classify_priority", classify_priority)
builder.add_node("generate_final_report", generate_final_report)

builder.set_entry_point("extract_topics")

builder.add_edge("extract_topics", "summarize_meeting")
builder.add_edge("summarize_meeting", "extract_action_items")

builder.add_conditional_edges(
    "extract_action_items",
    check_action_items,
    {
        "has_items": "classify_priority",
        "no_items": "generate_final_report"
    }
)

builder.add_edge("classify_priority", "generate_final_report")
builder.add_edge("generate_final_report", END)

graph = builder.compile()
```

Once compiled, running the whole pipeline is a single call: `graph.invoke(initial_state)`. This replaced the manual, step-by-step chaining I was doing during isolated testing (`state_after_topics = extract_topics(...)`, then feeding that into the next function by hand, and so on) — LangGraph handles all of that sequencing, including checking the conditional router at exactly the right moment, automatically.

12. Testing the Pipeline

I tested in stages rather than jumping straight to the full graph:

1. Each agent individually, using the sample transcript from the original project brief, to confirm basic correctness before anything was wired together.
2. The full compiled graph, on the same sample transcript, to confirm the automatic sequencing worked identically to my manual chaining.
3. A real, more complex transcript — a team meeting about an Android app crashing on login, with four speakers and far more action items than the sample. This was a genuinely useful stress test: it confirmed the prompts generalized to messier, more technical, real-world conversation rather than being narrowly tuned to the brief's example. Every action item was correctly attributed to the right role, and the priority

was correctly classified as High, matching the actual urgency expressed in the transcript ("please prioritize this issue today").

4. The no-action-items edge case, to confirm the conditional branch actually worked in both directions, not just the "items found" path I'd been testing by default.

13. Building the Streamlit Interface

The command-line version worked, but it wasn't something anyone besides me could actually use. I built `app.py` as a Streamlit front-end, and iterated on it a few times based on how it actually looked once running.

The interface lets someone either paste a transcript directly, upload a `.txt` file, or load a sample with one click, then press "Analyze Meeting" to run the full graph and see the results laid out as: topic tags, a summary paragraph, action items as individual cards with the owner shown underneath each task, and a colored priority badge (red for High, orange for Medium, green for Low). I went through a couple of rounds of visual revisions — starting with a light, minimal look, then moving to a dark, more "developer tool" aesthetic with a purple/cyan accent palette, monospace typography for labels, and a subtle glow effect on the priority badges.

One real bug I hit along the way: after adding the dark theme, the transcript text box background went dark but the text stayed dark too, making it unreadable. The fix was making sure text color was explicitly set (not just background color) with sufficient CSS specificity to override Streamlit's defaults. I also learned that Streamlit only reads its `.streamlit/config.toml` theme file once, at server startup — editing it while the app is still running doesn't apply until you fully stop and restart the server, not just save the file.

14. Version Control and Deployment

I initialized a proper Git repository for the project rather than treating version control as an afterthought. A `.gitignore` file excludes the virtual environment folder (`venv/`, which is large and fully reproducible from `requirements.txt`) and, critically, the `.env` file containing my real OpenAI API key — since public GitHub repositories are actively scanned for exposed credentials, and committing that file would be a real security mistake, not just a style issue.

The README documents what the project does, why it's architected as a multi-agent graph rather than a single prompt, the project's file structure, and setup instructions for anyone cloning the repository. For deployment to Streamlit Community Cloud, the API key is provided through Streamlit's built-in secrets management rather than being present in the repository at all, keeping the same "never commit real credentials" principle intact even in the deployed version.

15. A Full Walkthrough — Watching the State Change Step by Step

I think the architecture is easiest to actually understand by watching one real transcript move through it, so here's the Android login-crash meeting traced through the pipeline, stage by stage.

Starting state, right after the transcript is loaded:

```python
{
  "transcript": "Product Manager: Good morning everyone...",
  "topics": [],
  "summary": "",
  "action_items": [],
  "priority": "",
  "final_report": ""
}
```

Every field except `transcript` is empty — this is the "baton" at the very start of the relay.

After `extract_topics` runs, the model reads the whole transcript and returns a numbered list, which gets parsed into:

```python
"topics": [
  "Mobile app crashing during login",
  "Root cause investigation",
  "Authentication API changes",
  "Android-specific testing",
  "Communication with affected users"
]
```

Nothing else in the state has changed yet — `summary`, `action_items`, and `priority` are still empty. This is exactly why testing each node in isolation was useful: I could print the state after just this one node and confirm it was doing its job correctly before moving on.

After `summarize_meeting` runs, the `summary` field fills in with a short paragraph describing the crash reports, the suspicion that a recent authentication service update is the cause, and the team's decision to investigate both the mobile and backend sides. The `topics` field from the previous step is untouched — this node only had permission (by design, through its prompt) to write to `summary`.

After `extract_action_items` runs, the `action_items` list fills with entries like:

```python
{"task": "Review the Android login module for missing error handling", "owner":
{"task": "Prepare a detailed bug report with screenshots and logs", "owner": "Q
{"task": "Verify the authentication API for breaking changes", "owner": "Backen
```

and several more — nearly every speaker in this transcript commits to at least one task, which made it a good stress test compared to the shorter sample transcript.

The conditional check fires here. `check_action_items` looks at the list above, sees it's non-empty, and returns `"has_items"`. LangGraph routes execution to `classify_priority` rather than skipping straight to the final report.

After `classify_priority` runs, `priority` is set to `"High"` — correctly reflecting language in the transcript like "we must resolve this issue urgently" and "please prioritize this issue today," even though no one explicitly said the word "high priority."

Finally, `generate_final_report` runs, reading every field that's now populated and assembling the finished report string, which becomes the value returned to whoever called `graph.invoke()`.

I found tracing through an example like this far more useful for actually understanding LangGraph than reading documentation in the abstract — seeing the same dictionary object get progressively filled in, field by field, by independent functions, is really the entire mental model.

16. User Guide — How to Actually Use the App

For anyone opening the deployed app for the first time:

1. Go to the live app link: https://ai-meeting-notes-analyzer-sihag.streamlit.app/
2. You have three ways to provide a transcript: paste text directly into the box, upload a `.txt` file using the uploader, or click "Use sample transcript" to try it instantly without needing your own data.
3. Click **Analyze Meeting**. A spinner appears while the four LLM calls run in sequence — this typically takes a few seconds, not instant, since each agent is a separate model call rather than one fast lookup.
4. Once finished, the results appear in order: topics as small tags, a summary paragraph, action items as individual cards (each showing who owns it), and a colored priority badge.
5. If you want the plain-text version instead of the styled view, expand "View plain-text report," and there's a download button to save the report as a `.txt` file.

If a transcript genuinely has no action items — for example, a purely informational update with nothing anyone committed to doing — the app will correctly show "No action items identified in this meeting" and mark the priority as "Not applicable," rather than forcing a misleading classification onto an empty list.

17. Reflections on Prompt Engineering

A pattern I noticed repeatedly while building this: getting the *content* of a prompt roughly right was usually the easy part — GPT is generally good at reading a transcript and identifying topics or tasks. The harder, more iterative part was constraining the **output format** tightly enough that my Python code could reliably parse it afterward.

A few concrete examples of this from the project:

- Telling the model to return "ONLY a numbered list, nothing else" for topics, because without that constraint it would sometimes add a conversational preamble that broke the parsing logic.
- Choosing a pipe character (`|`) as the delimiter between a task and its owner, specifically because it's a character that almost never appears naturally inside an English sentence, unlike a comma.
- Asking for the literal word `NONE` when there are no action items, rather than trying to infer "emptiness" from a vague or blank-ish response, which would have been much less reliable to detect in code.
- Explicitly telling the summary agent *not* to include action items, because the model has access to the full transcript regardless of which agent is asking, and will blend tasks into a summary unless told otherwise.

I think this is a genuinely transferable lesson beyond this specific project: when an LLM's output needs to be consumed by code afterward, the prompt has to do double duty — get the reasoning right, *and* produce something a parser can trust.

18. Challenges, Lessons Learned, and Future Scope

The biggest technical lesson from this project wasn't really about LangGraph syntax — it was about **decomposition**: learning to break one fuzzy, multi-part task into small, independently testable pieces, each with a narrow prompt and a clear contract for what it reads and writes. That mindset showed up repeatedly, from the agent design itself down to how I structured the project's files.

The environment setup issue with the copied virtual environment was a good reminder that infrastructure problems can look identical to code problems on the surface (both just produce an error), but need a completely different kind of debugging — reading the actual file paths in an error message rather than assuming the bug is in my own logic.

If I extended this project further, the most useful next steps would be: per-speaker action item summaries, support for longer transcripts that exceed a single context window (chunking and merging results), exporting reports directly to PDF or a calendar/task-tracker integration, and a lightweight evaluation set to systematically test prompt changes against a range of transcript styles rather than relying on manual spot-checks.

This project and its documentation were completed with guidance from Claude (Anthropic), used as a learning and debugging assistant throughout the build process. All implementation, testing, and deployment decisions were carried out by me.